Detaillierte Testergebnisse für Testdurchlauf 1

1. *ZU BESTÄTIGENDE ERKLÄRUNGEN* [ID: 184402]

Während der tatsächlichen Klausur besteht ebenfalls die Möglichkeit, mit einer Aufsichtsperson zu sprechen, um Hilfestellung zu erhalten wenn eine Formulierung bei den Aufgaben missverständlich erscheint o.ä.

- Die Aufgaben müssen selbst gelöst werden, d.h. dabei ist fremde Hilfe verboten.
- Während der Probeklausur erreichen Sie die Aufsicht über den Meetings-Tab (der Raum heißt "Probeklausuraufsicht" und ist der unterste in der linken Spalte auf diesem Tab)
- Programmierlösungen müssen in den Klausureditor eingefügt werden mittels Klick auf das Büroklammersymbol, Hochladen und Speicherung durch Wechsel zu anderer Aufgabe. Sie sind selbst verantwortlich für das Speichern Ihrer Eingaben.
- Die Arbeit an der Klausur ist **spätestens 1min vor Timer-Ablauf** zu beenden und nochmals zu speichern. Andernfalls riskieren Sie Datenverlust bei Timerablauf, ohne Möglichkeit der Rekonstruktion.

◉ Ich habe die Erklärung gelesen und verstanden. Ich erkläre, dass ich nur zugelassene Hilfsmittel verwende. Mir ist bewusst, dass die Arbeit mit Software zur Plagiatsprävention überprüft werden kann.

2. Einige Java-Grundkenntnisse (I1-ID: v969emj0ajh0) [ID: 184403]

Aufgabenteile sind unabhängig zu beantworten und werden unabhängig bewertet.

1. Von klein nach groß ordnen (0.25P für jede richtige Position)

   Ordnen Sie VON OBEN NACH UNTEN die Werte vom kleinsten bis zum größten, d.h. **wenn X kleiner als Y ist, so muss X weiter oben stehen als Y**. Achten Sie auch auf Minuszeichen!

   Integer.MIN_VALUE (am kleinsten)

   - Integer.MAX_VALUE

   Double.MIN_VALUE

   Float.MIN_VALUE

   - Short.MIN_VALUE

   2 Milliarden

   Integer.MAX_VALUE

   7.8e9 (am größten)

2. Intervalltreffer (0.5P für jede korrekte Angabe)

   Füllen Sie die Lücken, so dass das Programm mit ganzen Zahlen A und B (A ist die kleinere) als Kommandozeilenparametern aufgerufen Folgendes leistet: Nach Einlesen von 10 Werten von der Standardeingabe wird die Anzahl der eingelesenen Werte ausgegeben, die zwischen A und B (einschließlich) liegen.

   (0.5P für jede korrekte Angabe)

```
class Intervalltreffer {
    public static void main(String[] args) {
        int


A = Integer.parseInt(args[0])




;
        int


B = Integer.parseInt(args[1])




;
        int


n = 0




;
        for (


int i = 0; i < 10; i++




) {
            int


z = StdIn.readInt()




;
            if (


z >= A && B >= z




)
```

```
n++




;
        }


System.out.println(n)




;
    }
}
```

3. Was kommt raus? (1P für die richtige Antwort).

   Beim Compilieren eines Java-Programms entsteht Bytecode .

3. Schleifen-Fehler (I1-ID: z8g7z8p0ufg0) [ID: 184404]

Die ungeraden natürlichen Zahlen sollen nacheinander aufsummiert werden bis die Summe größer als 24 ist. Untenstehende Klasse dafür ist jedoch sehr ungeschickt und dadurch sogar fehlerhaft programmiert.

```
 1:  public class Teilsumme {
 2:      public static void main(String[] args) {
 3:          int s = 0, n = 0;
 4:          boolean istGerade = true;
 5:          while ( s <= 24) {
 6:              if (istGerade = false)
 7:                  s += n;
 8:              n++;
 9:              istGerade = !istGerade;
10:          }
11:          System.out.println(s);
12:      }
13:  }
```

Erzeugt die Klasse

- einen Compilezeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-): -
- eine Endlosschleife? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-): 6
- eine RuntimeException ("Programm stürzt ab")? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-): -

Hinweis: Sie müssen für jede der drei Fragen die richtige Auswahl treffen, um volle Punktzahl zu erhalten.

4. Elementare Daten (I1-ID: 93rdw3918rg0c) [ID: 184405]

```
public static void a() {
    char a = 'a';
    a++;
    int b = (int) a;
    b++;
    System.out.print( (char) ('A' + b - a));
}
public static boolean getTrue() {
    System.out.print("T");
    return true;
}
public static boolean getFalse() {
    System.out.print("F");
    return true;
}
public static void q(int i) {
    int j = 43;
    System.out.print("G");
    if ( i / j > 0) System.out.print("Q");
    else System.out.print("R");
}
public static void d() {
    double dd = Double.MIN_VALUE;
    dd *= 2;
    dd -= Double.MIN_VALUE;
    dd /= 2;
    System.out.print(dd);
}
```

Welche Ausgaben bewirken diese Aufrufe, nacheinander in der `main`-Methode der gleichen Klasse ausgeführt?:

- `a();` B
- `boolean val = getTrue() || getFalse();` T
- `val = getTrue() && !getTrue();`
- `q(42);` GR
- `d();` 0.0

5. Integer-Arithmetik (I1-ID: zzfd4k81gdg0) [ID: 184406]

Sei `c` eine `int`-Variable mit dem Wert `Integer.MAX_VALUE` (also 2147483647). Welche Textausgabe erzeugen diese Anweisungen?:

```
System.out.println(2147483646-c); //


-1




System.out.println((-1)*c);       //


-2147483647




System.out.println(c+2);          //


-2147483647




System.out.println(c*2-1);        //


-3
```

6. Lokale Variablen (I1-ID: akldnih0djh0) [ID: 184408]

```
class LokaleVariablen {
    public static void d1(int n) {
        n = n + n;
    }
    public static void d2(int[] f, int n) {
        f[n] = f[n] + f[n];
    }
    public static void main(String[] args) {
        int n = 2;
        int[] f = { n, n, n};
        for (int i = 0; i < 3; i++) {
            d1(n);
            d2(f,n);
        }
        System.out.println(n);
        System.out.println(f[n]);
    }
}
```

Wie lautet die erste Ausgabezeile? 2

Wie lautet die zweite Ausgabezeile? 16

7. Mystery-Rekursionen (I1-ID: cyub8r30gvh0) [ID: 184409]

Teile A und B sind unabhängig voneinander zu beantworten und werden auch unabhängig voneinander bewertet.

Teil A

Betrachten Sie folgende Java-Funktion.

```
1: public static int mystery(int a, int b) {
2:     if (a == 0) return b;
3:     if (b == 0) return a;
4:
5:     return mystery(a - b, b);
6: }
```

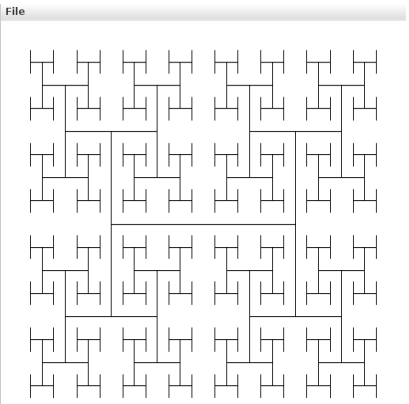
1. Was bewirkt der Aufruf `mystery(49,7)` ? Ergebnis 7 wird geliefert
2. Was bewirkt der Aufruf `mystery(7,49)` ? java.lang.StackOverflowError
3. Welcher Code muss in Zeile 4 eingefügt werden, damit jeder Aufruf mystery(a,b) mit a, b ≥0 den gleichen `int` -Wert liefert wie wie mystery(b,a)? if (a < b) return mystery(b, a);

Teil B

Der Aufruf `StdDraw.line(x0,y0,x1,y1)` zeichnet die Strecke zwischen Punkten `(x0,y0)` und `(x1,y1)` einer abstrakten Zeichenebene.

Dabei sind (x,y)-Koordinaten in üblicher Lage und der Punkt (0,0) links unten. Die Methode wird in nachfolgender Funktion benutzt.

```
 1: public static void draw(int n, double x, double y, double size) {
 2:     if (n == 0) return;
 3:     double x0 = x - size/2, x1 = x + size/2;
 4:     double y0 = y - size/2, y1 = y + size/2;
 5:     StdDraw.line( x0, y0, x0,  y1);
 6:     StdDraw.line( x1, y0, x1,  y1);
 7:     StdDraw.line( x0,  y, x1,  y);
 8:     // zeichne rekursiv 4 kleinere H-Baeume der Ordnung n-1
 9:     draw(n-1, x1, y1, size/2);
10:     draw(n-1, x0, y0, size/2);
11:     draw(n-1, x0, y1, size/2);
12:     draw(n-1, x1, y0, size/2);
13: }
```



Der „H-Baum" der Ordnung 4 auf dem Bild entsteht durch den Aufruf `draw(4, 0.5, 0.5, 0.5)` .

Dazu werden rekursiv an geeigneter Position vier H-Bäume (Teilbäume) nächstkleinerer Ordnung und Größe gezeichnet.

In welcher Reihenfolge werden die Teilbäume der Ordnung 3 fertig? oben rechts, unten links, oben links, unten rechts

8. Felder: Fehler oder ...? (I1-ID: 9je70sx0ejh0) [ID: 184410]

Erzeugt untenstehende Klasse

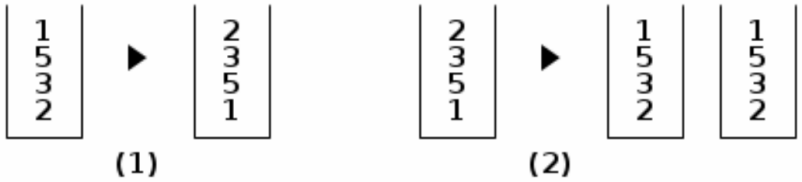
- einen Compilezeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-): -
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-): 6
- eine Ausgabe? Falls ja, so tragen Sie die Ausgabe ein, ansonsten einen Bindestrich (-): -

```
1:  class FehlerOderWas {
2:      public static void main(String[] args) {
3:          int[] a = { 2, 3, 5, 7, 11};
4:          int[] b = new int[6];
5:          b = a;
6:          a[0] = b[5];
7:          System.out.println(8+a[0] % 2);
8:      }
9:  }
```

9. Einen Stack kopieren (I1-ID: 36i3ih81hvh0) [ID: 184411]

Betrachten Sie folgende API zum Speichern von `int`-Werten (alle Methoden/Konstruktoren `public`) und die Grafik zur Illustration der Idee:

`public class Stack`	
`Stack()`	einen neuen Stack erzeugen
`boolean empty()`	testen, ob dieser Stack leer ist
`void push(int i)`	`i` in diesem Stack speichern
`int pop()`	letztgespeicherten Wert liefern und entfernen



Programmieren Sie eine weitere Instanzmethode:

```
public Stack copy() {
  // Rumpf (Ihre Aufgabe)
}
```

Spezifikation: Aufruf liefert Stack gleichen Inhalts wie dieser. Original und Kopie sind unabhängig benutzbar (Speichern beim Original ändert nicht die Kopie usw.).

Idee: (1) alle Werte aus diesem Stack entfernen und anderswo speichern, (2) von dort die Werte wieder entfernen und in diesem und im neuem Stack speichern.

Stack h = new Stack(); Stack c = new Stack();

while (!this.empty()) {

h.push(this.pop()); }

while (!h.empty()) {

int i = h.pop();

this.push(i); c.push(i);}

return c;

10. OOP-Verständnisfragen (I1-ID: 0kails31hvh0) [ID: 184412]

Java bietet mehrere für die Datenkapselung notwendige Mechanismen, zum Beispiel Instanzmethoden

Konstruktoraufrufe lösen eine Reihe von Aktionen aus, zum Beispiel Instanzvariablen setzen

Welche Erklärung trifft auf den Begriff "Überschreiben" zu?

eine von der Basisklasse geerbte Methode neu definieren

Welche dieser Situationen umschreibt man mit dem Begriff "Dynamische Bindung"?

bei Aufruf einer Instanzmethode steht der auszuführende Code erst zur Laufzeit fest

11. Laufzeitbetrachtungen (I1-ID: qgrk5ck0kvh0) [ID: 184413]

1. Was ist die Kernoperation bei der Methode `a(int n, boolean f)` ? Geben Sie die Zeilennummer an. 8

```
 1:         public static int a(int n, boolean f) {
 2:             if (n < 0)
 3:                 return 0;
 4:             int s = 0;
 5:             for (int i = 1; i <= n; i++)
 6:                 for (int j = 0; j < i; j++)
 7:                     if ( f )
 8:                         s += 1;
 9:             return s;
10:         }
```

- Geben Sie die worst-case-Anzahl der Kernoperationen von `a(int n, boolean f)` in Abhängigkeit vom Argument `n` in Tilde-Notation an. ~0.5n²
- Ergänzen Sie die Methode `b(int n, boolean f)` so, dass sie genau das gleiche leistet wie `a(int n, boolean f)` , dabei aber eine Laufzeit konstanter Wachstumsordnung hat.

```
public static int b(int n, boolean f) {
    if (n < 0 || !f)
        return 0;
    return


n*(n+1)/2;









    }
```

12. Klasse Punkt (I1-ID: esidz5x0kvh1) [ID: 184416]

Ein Punkt der Ebene ist durch seine Koordinaten (zwei `double` -Werte) definiert. Schreiben Sie eine Java-Klasse für unveränderliche Punkt-Objekte, die folgende API realisiert:

public class Punkt	
public Punkt(double x, double y)	erzeugt neues Punkt-Objekt
public static int anzahl()	liefert die Anzahl erzeugter Punkt-Objekte
public double getX()	liefert die x-Koordinate dieses Punktes
public boolean equals(Punkt other)	testet, ob dieser Punkt und other die gleiche Position haben
public String toString()	String-Umwandlung

Fügen Sie einen Test-Client hinzu, der eine ganze Zahl N von der Kommandozeile übernimmt und genau folgendes leistet: Koordinaten von N Punkten aus der Standardeingabe lesen, die Punkte zeilenweise ausgeben (keinen weiteren Text) und testen, ob darunter ein Paar von Punkten mit gleichen Koordinaten vorkommt und das Ergebnis des Tests (nur true oder false auf einer neuen Zeile) ausgeben.

Beispiel: der Aufruf `echo 0.0 0.1 1.0 1.1 0.0 0.1 | java Punkt 3` erzeugt exakt die Ausgabe

```
Punkt(0.0,0.1)
Punkt(1.0,1.1)
Punkt(0.0,0.1)
true
```

Gehen Sie davon aus, dass ein korrekter Aufruf erfolgt, insbesondere N>0 ist. Dies soll nicht getestet werden.

- Verwenden Sie exakt die angegebenen Bezeichner und das vorgegebene Ausgabeformat.
- Verwenden Sie zum Einlesen der Koordinaten die Bibliothek `StdIn` .
- Achten Sie auf Codeeinrückung.
- Machen Sie die Unveränderlichkeit der Objekte am Code deutlich.
- Machen Sie keine unnötige Methodenaufrufe und definieren Sie keine nicht geforderten Methoden.

```
1  public class Punkt {
2      private final double x;
3      private final double y;
4
5      private static int anzahl = 0;
6
7      public Punkt(double x, double y){
8          this.x = x;
9          this.y = y;
10
11         anzahl++;
12     }
13
14     public static int anzahl(){
15         return anzahl;
16     }
17
18     public double getX(){
19         return this.x;
20     }
21
22     public boolean equals(Punkt other){
23         return this.x == other.x && this.y == other.y;
24     }
25
26     public String toString(){
27         return "Punkt(" + this.x + "," + this.y + ")";
28     }
29
30     public static void main(String[] args){
31         int N = Integer.parseInt(args[0]);
32         Punkt[] p = new Punkt[N];
33
34         for(int i = 0; i < N; i++){
35             p[i] = new Punkt(StdIn.readDouble(), StdIn.readDouble());
```

	0:0
	0:0

13. Ein weiteres Wort in eine Suchsel-Lösung eintragen (I1-ID: 85reico093k0) [ID: 184417]

"Suchsels" bestehen aus einer $N \times N$ - Anordnung von Buchstaben und einer Liste von Suchwörtern, die darin von links nach rechts oder von oben nach unten zu lesen sind. Hier ein Beispiel (mit Zeilen- und Spaltennummern):

```
Lösung                          Rätsel

   0 1 2 3                         0 1 2 3
0: F                            0: F B B X
1: E N T E                      1: E N T E
2: I                            2: I U E K
3: N O T E                      3: N O T E
```

Die Lösung besteht aus einem zweidimensionalen `char` - Array, das die Buchstaben der Suchwörter an den richtigen Stellen enthält, ansonsten nur Leerzeichen. Im Rätsel sind die Leerzeichen zufällig durch Buchstaben ersetzt. In die Lösung passen aber noch weitere Wörter, z.B. `FILZ` (ab Position (0,0) waagerecht) und `INFO` (ab (0,1) senkrecht).

Aufgabe

Programmieren Sie eine statische Methode

```
public static boolean insertLR(String w, int x, int y, char[][] s)
```

die (1) prüft, ob `w` ab Position `(x,y)` von links nach rechts in die in `s` gespeicherte Lösung eingetragen werden kann, ohne bereits vorhandene Buchstaben zu überschreiben (Leerzeichen können überschrieben werden). Ist das der Fall, so wird (2) das Wort "eingetragen" (also der Inhalt von `s` entsprechend geändert). Im Erfolgsfall wird `true` zurückgeliefert, ansonsten `false`.

Hinweise

1. Sie müssen unter anderem mittels `w.length()` überprüfen, ob `s` groß genug ist, um das Wort ab Position `(x,y)` aufzunehmen.
2. Auf die Zeichen von `w` können Sie mit `w.charAt(int pos)` zugreifen (siehe Skript Seite 168f.).
3. Tragen Sie die komplette Methode in untenstehenden "Klausureditor" ein. Sie brauchen sie nicht in eine Klasse einbetten.

```
1  public static boolean insertLR(String w, int x, int y, char[][] s) {
2      if(x + w.length() > s[y].length){
3          return false;
4      }
5
6      for(int i = 0, i < w.length(); i++){
7          if(s[y][x + i] != ' ' && s[y][x + i] != w.charAt(i)){
8              return false;
9          }
10     }
11
12     for(int i = 0; i < w.length(); i++){
13         s[y][x + i] = w.charAt(i);
14     }
15
16     return true;
17 }
```